

A pod is broken. The fix is **slow, manual, and repetitive**

- ✗ Every incident is the same dance: `get pods` → `describe` → `logs` → guess the cause.
- ✗ The failures rhyme — **CrashLoopBackOff, ImagePullBackOff, OOMKilled, Unschedulable** — but you re-diagnose from scratch each time.
- ✗ Deep expertise lives with **a few senior engineers**; everyone else is blocked or pages them at 2am.
- ✗ Under pressure people **fix the symptom** (restart it) not the cause — and it breaks again.

The gap: the *reading and reasoning* is slow and mechanical — but the *deciding* must stay with a human. We want to automate the first without giving up the second.

An agent that **diagnoses** — and asks before it acts

A **Kubernetes Troubleshooting Agent**: point it at a namespace, it finds the broken pods, reads their status + events + logs, and gpt-4o returns the *real root cause* with the evidence — then proposes one fix and waits for a human to approve it.

01

See

Read-only: pods, container states, events, log tails.

02

Diagnose

gpt-4o names the failure, cites the evidence line, explains why.

03

Propose

One safe fix — restart / scale / delete-pod — or "manual".

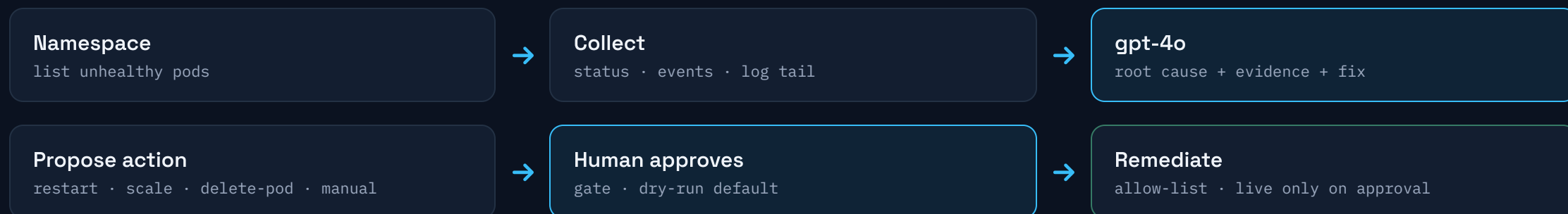
04

Gate

Human approves; dry-run by default before anything runs.

The agent proposes; a human disposes. The LLM only reads and reasons — the one place that can change a cluster is a tiny, safe allow-list behind an explicit approval.

The **process**, end to end



Connects to: OpenAI (gpt-4o) · your cluster via the Kubernetes client (kubeconfig, read-only). Runs on a realistic **mock cluster** with zero setup, or a real one with `USE_REAL_K8S=true` + `rbac.yaml`. Launch: `streamlit run app.py`.